缓存组是一个命名空间，管理特定类别数据的缓存，提供了数据的获取、设置、删除等基本操作，同时负责缓存未命中时的数据加载和分布式节点间的数据同步。

该模块将缓存、分布式通信和数据加载策略融为一体，通过单飞（singleflight）机制防止缓存击穿，通过分布式协议保持数据一致性，通过统计指标监控缓存效率，是整个 LCache 系统的中枢组件。

核心结构设计

全局缓存组管理

代码块

```
var (
    groupsMu sync.RWMutex
    groups   = make(map[string]*Group)
)
```

使用全局映射表管理所有缓存组，通过读写锁保证并发安全：

- `groups`：名称到缓存组的映射
- `groupsMu`：保护映射的读写锁

数据加载接口

代码块

```
// Getter 加载键值的回调函数接口
type Getter interface {
    Get(ctx context.Context, key string) ([]byte, error)
}

// GetterFunc 函数类型实现 Getter 接口
type GetterFunc func(ctx context.Context, key string) ([]byte, error)

// Get 实现 Getter 接口
func (f GetterFunc) Get(ctx context.Context, key string) ([]byte, error) {
    return f(ctx, key)
}
```

`Getter` 接口定义了缓存未命中时从数据源加载数据的方法：

- 接受上下文和键作为输入
- 返回字节切片和可能的错误

缓存组结构

代码块

```go
// Group 是一个缓存命名空间
type Group struct {
    name       string
    getter     Getter
    mainCache  *Cache
    peers      PeerPicker
    loader     *singleflight.Group
    expiration time.Duration // 缓存过期时间，0表示永不过期
    closed     int32         // 原子变量，标记组是否已关闭
    stats      groupStats    // 统计信息
}

// groupStats 保存组的统计信息
type groupStats struct {
    loads        int64 // 加载次数
    localHits    int64 // 本地缓存命中次数
    localMisses  int64 // 本地缓存未命中次数
    peerHits     int64 // 从对等节点获取成功次数
    peerMisses   int64 // 从对等节点获取失败次数
    loaderHits   int64 // 从加载器获取成功次数
    loaderErrors int64 // 从加载器获取失败次数
    loadDuration int64 // 加载总耗时（纳秒）
}
```

`Group` 结构体是整个缓存系统的核心，包含以下关键字段：

- `name`：缓存组名称，唯一标识
- `getter`：数据加载回调，缓存未命中时调用
- `mainCache`：本地缓存实例
- `peers`：分布式节点选择器
- `loader`：单飞组，防止缓存击穿
- `expiration`：缓存过期时间
- `closed`：组是否已关闭标志
- `stats`：统计信息

核心功能实现

数据获取

代码块

```go
// Get 从缓存获取数据
func (g *Group) Get(ctx context.Context, key string) (ByteView, error) {
    // 检查组是否已关闭
    if atomic.LoadInt32(&g.closed) == 1 {
        return ByteView{}, ErrGroupClosed
    }

    if key == "" {
        return ByteView{}, ErrKeyRequired
    }

```

```go
    // 从本地缓存获取
    view, ok := g.mainCache.Get(ctx, key)
    if ok {
        atomic.AddInt64(&g.stats.localHits, 1)
        return view, nil
    }

    atomic.AddInt64(&g.stats.localMisses, 1)

    // 尝试从其他节点获取或加载
    return g.load(ctx, key)
}
```

数据获取流程：

1. **状态检查**：验证组是否处于活跃状态
2. **参数验证**：确保键不为空
3. **本地缓存**：尝试从本地缓存获取数据
4. **统计更新**：记录本地命中或未命中
5. **加载数据**：缓存未命中时加载数据

这个流程体现了缓存的基本思想：先查本地，未命中则加载。同时通过原子操作安全地更新统计数据。

数据加载

代码块

```go
// load 加载数据
func (g *Group) load(ctx context.Context, key string) (value ByteView, err error) {
    // 使用 singleflight 确保并发请求只加载一次
    startTime := time.Now()
    viewi, err := g.loader.Do(key, func() (interface{}, error) {
        return g.loadData(ctx, key)
    })

    // 记录加载时间
    loadDuration := time.Since(startTime).Nanoseconds()
    atomic.AddInt64(&g.stats.loadDuration, loadDuration)
    atomic.AddInt64(&g.stats.loads, 1)

    if err != nil {
        atomic.AddInt64(&g.stats.loaderErrors, 1)
        return ByteView{}, err
    }

    view := viewi.(ByteView)

    // 设置到本地缓存
    if g.expiration > 0 {
        g.mainCache.AddWithExpiration(key, view, time.Now().Add(g.expiration))
    } else {
        g.mainCache.Add(key, view)
    }

    return view, nil
```

```go
}

// loadData 实际加载数据的方法
func (g *Group) loadData(ctx context.Context, key string) (value ByteView, err error) {
    // 尝试从远程节点获取
    if g.peers != nil {
        peer, ok, isSelf := g.peers.PickPeer(key)
        if ok && !isSelf {
            value, err := g.getFromPeer(ctx, peer, key)
            if err == nil {
                atomic.AddInt64(&g.stats.peerHits, 1)
                return value, nil
            }

            atomic.AddInt64(&g.stats.peerMisses, 1)
            logrus.Warnf("[LCache] failed to get from peer: %v", err)
        }
    }

    // 从数据源加载
    bytes, err := g.getter.Get(ctx, key)
    if err != nil {
        return ByteView{}, fmt.Errorf("failed to get data: %w", err)
    }

    atomic.AddInt64(&g.stats.loaderHits, 1)
    return ByteView{b: cloneBytes(bytes)}, nil
}

// getFromPeer 从其他节点获取数据
func (g *Group) getFromPeer(ctx context.Context, peer Peer, key string) (ByteView, error) {
    bytes, err := peer.Get(g.name, key)
    if err != nil {
        return ByteView{}, fmt.Errorf("failed to get from peer: %w", err)
    }
    return ByteView{b: bytes}, nil
}
```

数据加载的多层次流程：

1. **单飞机制**：使用 singleflight 防止缓存击穿
2. **性能监控**：记录加载时间和次数
3. **分布式查询**：先尝试从其他节点获取
4. **源数据加载**：如果分布式查询失败，从数据源加载
5. **结果缓存**：将加载的数据设置到本地缓存
6. **统计更新**：记录各类加载结果的统计信息

这个设计充分体现了分布式缓存的优势：利用整个集群的缓存能力，减轻单个节点的负担，同时降低对原始数据源的访问压力。

数据设置

代码块

```go
// Set 设置缓存值
```

```go
func (g *Group) Set(ctx context.Context, key string, value []byte) error {
    // 检查组是否已关闭
    if atomic.LoadInt32(&g.closed) == 1 {
        return ErrGroupClosed
    }

    if key == "" {
        return ErrKeyRequired
    }
    if len(value) == 0 {
        return ErrValueRequired
    }

    // 检查是否是从其他节点同步过来的请求
    isPeerRequest := ctx.Value("from_peer") != nil

    // 创建缓存视图
    view := ByteView{b: cloneBytes(value)}

    // 设置到本地缓存
    if g.expiration > 0 {
        g.mainCache.AddWithExpiration(key, view, time.Now().Add(g.expiration))
    } else {
        g.mainCache.Add(key, view)
    }

    // 如果不是从其他节点同步过来的请求，且启用了分布式模式，同步到其他节点
    if !isPeerRequest && g.peers != nil {
        go g.syncToPeers(ctx, "set", key, value)
    }

    return nil
}
```

数据设置流程：

1. **状态检查**：验证组是否已关闭
2. **参数验证**：确保键和值都有效
3. **请求来源检查**：区分本地请求和远程同步请求
4. **本地缓存更新**：设置到本地缓存
5. **分布式同步**：本地请求时异步同步到其他节点

请求来源区分很重要，它防止了缓存操作的无限循环传播，确保每个更新只在集群中同步一次

分布式操作同步

代码块

```go
// syncToPeers 同步操作到其他节点
func (g *Group) syncToPeers(ctx context.Context, op string, key string, value []byte) {
    if g.peers == nil {
        return
    }

    // 选择对等节点
```

```
    peer, ok, isSelf := g.peers.PickPeer(key)
    if !ok || isSelf {
        return
    }

    // 创建同步请求上下文
    syncCtx := context.WithValue(context.Background(), "from_peer", true)

    var err error
    switch op {
    case "set":
        err = peer.Set(syncCtx, g.name, key, value)
    case "delete":
        _, err = peer.Delete(g.name, key)
    }

    if err != nil {
        logrus.Errorf("[LCache] failed to sync %s to peer: %v", op, err)
    }
}
```

分布式同步的关键流程：

1. **节点选择**：使用一致性哈希选择目标节点
2. **标记来源**：在上下文中标记这是一个同步请求
3. **操作分发**：根据操作类型调用相应的远程方法
4. **错误处理**：记录同步失败的日志

通过标记同步请求的来源，解决了分布式系统中的环形传播问题，确保每个操作只在集群中传播一次

功能特定分析

多级缓存架构

kamacache 实现了典型的多级缓存架构：

1. **本地缓存**：最快，但容量有限
2. **远程节点缓存**：次快，扩大了有效缓存容量
3. **数据源**：最慢，但提供完整数据

这种架构类似于 CPU 的缓存层次结构，通过空间换时间的策略提高数据访问速度。当数据在本地缓存中找不到时，会尝试从远程节点获取，进一步降低对原始数据源的访问频率。

防止击穿

缓存击穿是指热点数据过期瞬间，大量请求同时涌入数据源的现象。LCache 通过 singleflight 机制有效防止了这个问题：

代码块

```
viewi, err := g.loader.Do(key, func() (interface{}, error) {
    return g.loadData(ctx, key)
```

```
})
```

singleflight 确保对同一个键的并发请求只执行一次加载操作，其他请求等待并共享结果，这样可以：

1. 减轻数据源负担
2. 避免重复计算
3. 提高响应速度
4. 减少资源消耗